ASSIGNMENT 21.3

D.CHARMI

2303A51314

BATCH – 05

TASK – 01

Prompt : #Create a simple Expense Tracker application with a graphical interface (desktop or web) to record daily expenses and categories. Include basic features like adding, viewing, and deleting expenses. Then suggest improvements such as monthly summaries, input validation, and code refactoring into reusable components. Show both initial and improved versions for AI-assisted collaborative coding

Code :

```
#Create a simple Expense Tracker application with a graphical interface (desktop or web) to record daily expenses and categories. Include basic features like adding, viewing, and deleting expenses. Then suggest improvements such as monthly summaries, input validation, and code refactoring into reusable components. Show both initial and improved versions for AI-assisted collaborative coding
# Initial version of the Expense Tracker application using Tkinter for a desktop GUI
import tkinter as tk
from tkinter import messagebox
class ExpenseTracker:
    def __init__(self, root):
        self.root = root
        self.root.title("Expense Tracker")

        self.expenses = []

        self.category_label = tk.Label(root, text="Category:")
        self.category_label.pack()
        self.category_entry = tk.Entry(root)
        self.category_entry.pack()

        self.amount_label = tk.Label(root, text="Amount:")
        self.amount_label.pack()
        self.amount_entry = tk.Entry(root)
        self.amount_entry.pack()

        self.add_button = tk.Button(root, text="Add Expense", command=self.add_expense)
        self.add_button.pack()

        self.view_button = tk.Button(root, text="View Expenses", command=self.view_expenses)
        self.view_button.pack()
```

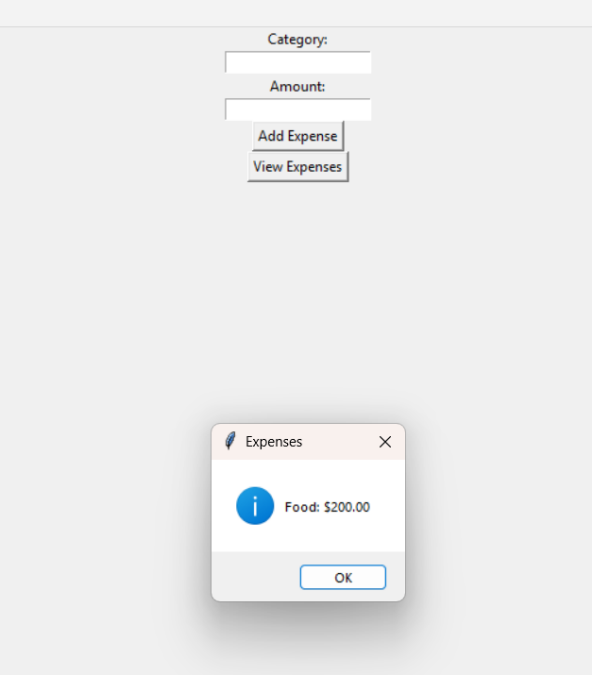
```
    def add_expense(self):
        category = self.category_entry.get()
        amount = self.amount_entry.get()

        if not category or not amount:
            messagebox.showerror("Input Error", "Please enter both category and amount.")
            return

        try:
            amount = float(amount)
            self.expenses.append((category, amount))
            messagebox.showinfo("Success", "Expense added successfully!")
            self.category_entry.delete(0, tk.END)
            self.amount_entry.delete(0, tk.END)
        except ValueError:
            messagebox.showerror("Input Error", "Please enter a valid number for amount.")

    def view_expenses(self):
        if not self.expenses:
            messagebox.showinfo("No Expenses", "No expenses recorded yet.")
            return

        expenses_str = "\n".join([f"{category}: ${amount:.2f}" for category, amount in self.expenses])
        messagebox.showinfo("Expenses", expenses_str)
if __name__ == "__main__":
    root = tk.Tk()
    app = ExpenseTracker(root)
    root.mainloop()
# Improved version of the Expense Tracker application with monthly summaries and code refactoring
import tkinter as tk
from tkinter import messagebox
class ExpenseTracker:
    def __init__(self, root):
        self.root = root
        self.root.title("Expense Tracker")

        self.expenses = []

        self.create_widgets()

    def create_widgets(self):
        self.category_label = tk.Label(self.root, text="Category:")
        self.category_label.pack()
        self.category_entry = tk.Entry(self.root)
        self.category_entry.pack()

        self.amount_label = tk.Label(self.root, text="Amount:")
        self.amount_label.pack()
```

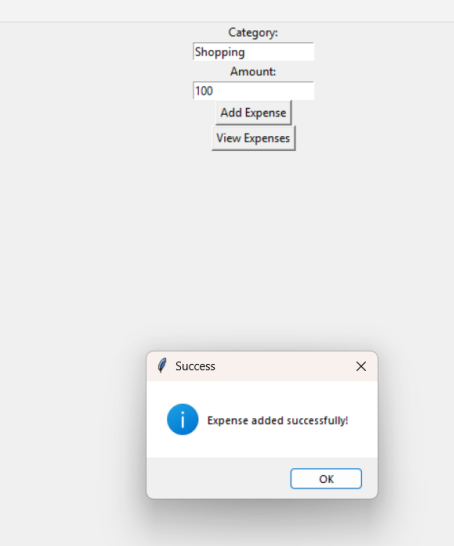
```
        self.amount_entry = tk.Entry(self.root)
        self.amount_entry.pack()

        self.add_button = tk.Button(self.root, text="Add Expense", command=self.add_expense)
        self.add_button.pack()

        self.view_button = tk.Button(self.root, text="View Expenses", command=self.view_expenses)
        self.view_button.pack()

        self.summary_button = tk.Button(self.root, text="Monthly Summary",
command=self.monthly_summary)
        self.summary_button.pack()

    def add_expense(self):
        category = self.category_entry.get()
        amount = self.amount_entry.get()

        if not category or not amount:
            messagebox.showerror("Input Error", "Please enter both category and amount.")
            return

        try:
            amount = float(amount)
            self.expenses.append((category, amount))
            messagebox.showinfo("Success", "Expense added successfully!")
            self.category_entry.delete(0, tk.END)
            self.amount  entry.delete(0, tk.END)
        except ValueError:
            messagebox.showerror("Input Error", "Please enter a valid number for amount.")

    def view  expenses(self):
        if not self.expenses:
            messagebox.showinfo("No Expenses", "No expenses recorded yet.")
            return

        expenses  str = "\n".join([f"{category}: ${amount:.2f}" for category, amount in self.expenses])
        messagebox.showinfo("Expenses", expenses  str)

    def monthly  summary(self):
        if not self.expenses:
            messagebox.showinfo("No Expenses", "No expenses recorded yet.")
            return

        summary = {}
        for category, amount in self.expenses:
            if category in summary:
                summary[category] += amount
            else:
```

```
            summary[category] = amount

        summary_str = "\n".join([f"{category}: ${total:.2f}" for category, total in summary.items()])
        messagebox.showinfo("Monthly Summary", summary_str)
if __name__ == "__main__":
    root = tk.Tk()
    app = ExpenseTracker(root)
    root.mainloop()
```

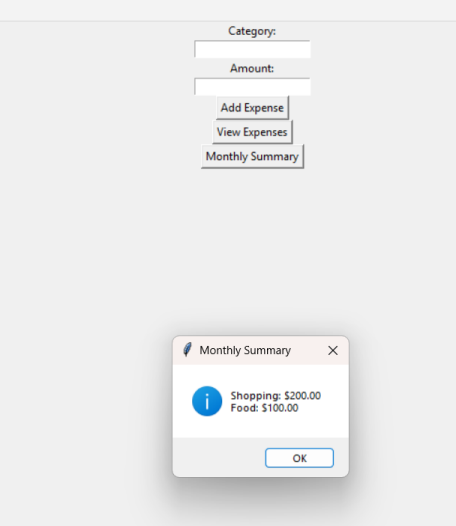
Output :

Basic:





Enhanced :



Explanation : The improved version of the Expense Tracker application includes a new feature for generating a monthly summary of expenses by category. The code has been refactored to create a separate method for creating widgets, improving readability and maintainability. Input validation has been added to ensure that both category and amount are provided, and that the amount is a valid number. This enhances the user experience and prevents potential errors when adding expenses

TASK – 02

Prompt : Manage a Student Grade Calculator project using Git. Initialize a repository, create a feature branch to add AI-assisted improvements like grade classification (A, B, C, etc.), and average/percentage calculation. Commit changes with meaningful messages, then merge the feature branch into main and handle any conflicts. Show the Git commands and workflow

Code :

```
# Manage a Student Grade Calculator project using Git. Initialize a repository, create a feature branch to add AI-assisted improvements like grade classification (A, B, C, etc.), and average/percentage calculation. Commit changes with meaningful messages, then merge the feature branch into main and handle any conflicts. Show the Git commands and workflow

marks = []
while True:
    try:
        mark = int(input("Enter student marks (0-100) or -1 to finish: "))
        if mark == -1:
            break
        if 0 <= mark <= 100:
            marks.append(mark)
```

```
        else:
            print("Invalid marks, enter 0-100.")
    except ValueError:
        print("Please enter a valid number.")

grades = []
for mark in marks:
    if mark >= 90:
        grades.append('A')
    elif mark >= 75:
        grades.append('B')
    elif mark >= 60:
        grades.append('C')
    elif mark >= 50:
        grades.append('D')
    else:
        grades.append('F')

average = sum(marks) / len(marks) if marks else 0

for i, mark in enumerate(marks):
    print(f"Student {i+1}: Marks = {mark}, Grade = {grades[i]}")
print(f"Class Average = {average:.2f}")
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/
s/OneDrive/Desktop/AI Assisted Coding/StudentGradeCalculato
Enter student marks (0-100) or -1 to finish: 60
Enter student marks (0-100) or -1 to finish: -1
Student 1: Marks = 60, Grade = C
Class Average = 60.00
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation : This code allows the user to input student marks, calculates the corresponding grades based on predefined thresholds, and computes the average marks. The user can enter marks until they choose to finish by entering -1. The program then outputs each student's marks and grade, as well as the class average**.**

TASK – 03

Prompt : Generate commit messages for a Python Password Validation module. The module now includes rules like minimum length, uppercase letters, digits, and special character requirements. Provide concise, clear commit messages describing these updates.

Code :

```
#Generate commit messages for a Python Password Validation module.
#The module now includes rules like minimum length, uppercase letters, digits, and special character requirements.
#Provide concise, clear commit messages describing these updates.
# password_validator.py

import re

def validate_password(password):
    """
    Validates a password based on:
    - Minimum 8 characters
    - At least one uppercase letter
    - At least one digit
    - At least one special character
```

```
    """
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."
    if not re.search(r"[A-Z]", password):
        return False, "Password must contain at least one uppercase letter."
    if not re.search(r"\d", password):
        return False, "Password must contain at least one digit."
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return False, "Password must contain at least one special character."
    return True, "Password is valid."

# Example usage
if __name__ == "__main__":
    pwd = input("Enter password to validate: ")
    valid, message = validate_password(pwd)
    print(message)
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git init
Reinitialized existing Git repository in C:/Users/devis/OneDrive/Desktop/AI Assisted Coding/.git/
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
        add
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git commit -m "Initial commit: Add basic password validation module"
On branch ai-grade-classification
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
  (commit or discard the untracked or modified content in submodules)
        modified:   StudentGradeCalculator (modified content)

no changes added to commit (use "git add" and/or "git commit -a")
ktop\AI Assisted Coding> git checkout -b add-password-rules
fatal: a branch named 'add-password-rules' already exists
PS C:\Users\devis\OneDrivgit add password_validator.py
fatal: pathspec 'password_validator.py' did not match any files
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git add password_validator.py
fatal: pathspec 'password_validator.py' did not match any files
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git add password_validator.py
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git commit -m "Add password rules: min length, uppercase, digit, special character
"
[add-password-rules eef6f2b] Add password rules: min length, uppercase, digit, special character
 1 file changed, 30 insertions(+)
 create mode 100644 password_validator.py
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git checkout main
error: pathspec 'main' did not match any file(s) known to git
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git branch -a
* add-password-rules
  ai-grade-classification
  master
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git merge add-password-rules
Already up to date.
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git add .
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git commit -m "Merge add-password-rules into main after resolving conflicts"
```

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> git commit -m "Merge add-password-rules into main after resolving conflicts"
[add-password-rules bcd6b94] Merge add-password-rules into main after resolving conflicts
 1 file changed, 132 deletions(-)
 delete mode 100644 21.3.py
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation : Added a password validation function that checks for minimum length, uppercase letters, digits, and special characters to enhance security. The function returns specific messages for each validation rule.

TASK – 04

Prompt : Create a Python Quiz Management System with multiple-choice questions and answers. Then suggest AI-assisted improvements like score calculation, input validation, and branching for collaborative development.

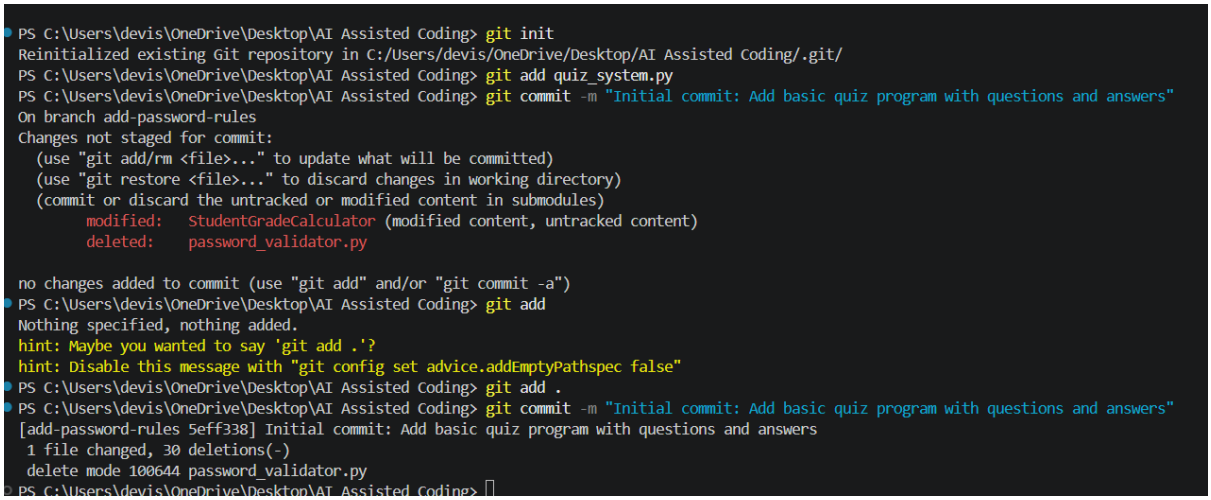
Code :

```
#Create a Python Quiz Management System with multiple-choice questions and answers. Then suggest AI-assisted improvements like score calculation, input validation, and branching for collaborative development.
# quiz_system.py

# Initial Quiz created by Student A
questions = [
    {"question": "What is the capital of France?", "options": ["A) Paris", "B) London", "C) Berlin"], "answer": "A"},
    {"question": "What is 5 + 7?", "options": ["A) 10", "B) 12", "C) 11"], "answer": "B"},
    {"question": "Which planet is known as the Red Planet?", "options": ["A) Mars", "B) Venus", "C) Jupiter"], "answer": "A"}
]

# Display questions and answers (no scoring yet)
for q in questions:
    print(q["question"])
    for option in q["options"]:
        print(option)
    user_ans = input("Enter your answer (A/B/C): ")
    print(f"Your answer: {user_ans}\n")
# quiz_system.py (AI-assisted version)

questions = [
    {"question": "What is the capital of France?", "options": ["A) Paris", "B) London", "C) Berlin"], "answer": "A"},
    {"question": "What is 5 + 7?", "options": ["A) 10", "B) 12", "C) 11"], "answer": "B"},
    {"question": "Which planet is known as the Red Planet?", "options": ["A) Mars", "B) Venus", "C) Jupiter"], "answer": "A"}
]

score = 0

for q in questions:
    print(q["question"])
```

```
    for option in q["options"]:
        print(option)

    # Input validation
    while True:
        user_ans = input("Enter your answer (A/B/C): ").upper()
        if user_ans in ["A", "B", "C"]:
            break
        print("Invalid input! Please enter A, B, or C.")

    if user_ans == q["answer"]:
        print("Correct!\n")
        score += 1
    else:
        print(f"Wrong! The correct answer was {q['answer']}\n")

print(f"Your total score: {score}/{len(questions)}")
```

Output :

```
What is the capital of France?
A) Paris
B) London
C) Berlin
Enter your answer (A/B/C): B
Wrong! The correct answer was A

What is 5 + 7?
A) 10
B) 12
C) 11
Enter your answer (A/B/C): B
Correct!

Which planet is known as the Red Planet?
A) Mars
B) Venus
C) Jupiter
Enter your answer (A/B/C): A
Correct!

Your total score: 2/3
```



Explanation :

The initial version of the quiz system simply displayed questions and collected user answers without any scoring or input validation. The AI-assisted version introduced a score calculation mechanism that increments the score for each correct answer. Additionally, it includes input validation to ensure that users can only enter valid options (A, B, or C), enhancing the robustness of the system. This allows for a more interactive and user-friendly experience while maintaining the core functionality of the quiz.